

# Laborationsrapport

**Projektuppgift / Webbapplikation (mashup)**

*DT211G, Frontend-baserad webbutveckling*

Författare: Riina Aaltonen, [riia2500@student.miun.se](mailto:riia2500@student.miun.se)

Termin, år: VT, 2026



**Mittuniversitetet**  
MID SWEDEN UNIVERSITY

# 1 Sammanfattning

Detta projekt består av en webbapplikation som gör det möjligt att söka, bläddra och filtrera filmer och TV-serier genom data från The Movie Database (TMDb) API. Applikationen är en mashup där filmdata kombineras med trailers från YouTube API för att skapa en mer komplett användarupplevelse.

Användaren kan utforska olika kategorier av innehåll, se detaljerad information om filmer och serier i en modal samt spara favoriter i en egen watchlist via localStorage där användaren även får möjlighet att lägga till egen data. Användaren kan lägga till en status på filmen, en egen rating samt en kommentar om filmen. Applikationen innehåller även funktioner för filtrering baserat på typ (film eller serie), genre samt sökfunktion i realtid.

Syftet med projektet är att skapa en interaktiv och dynamisk webbapplikation som visar hur externa API:er kan kombineras med frontend-logik för att skapa en användarvänlig tjänst för film- och serieinnehåll.

## 2 Innehållsförteckning

|                                                                |    |
|----------------------------------------------------------------|----|
| .....                                                          | 1  |
| 1 Sammanfattning .....                                         | 2  |
| 2 Innehållsförteckning.....                                    | 3  |
| 3 Terminologi .....                                            | 4  |
| 4 Inledning .....                                              | 5  |
| 4.1 Bakgrund och problemmotivering .....                       | 5  |
| 4.2 Konkreta mål .....                                         | 5  |
| 5 Metod .....                                                  | 6  |
| 6 Konstruktion.....                                            | 7  |
| 6.1 Designskisser och grafisk profil.....                      | 7  |
| 6.2 Hämtning och hantering av data .....                       | 7  |
| 6.3 Rendering av film- och seriekort.....                      | 8  |
| 6.4 Detaljerad information och modal .....                     | 8  |
| 6.5 Watchlist.....                                             | 8  |
| 6.6 Movie Journal.....                                         | 9  |
| 6.7 Kopplingen mellan TMDB-data och användarens egna data..... | 9  |
| Resultat.....                                                  | 11 |
| 7 Slutsatser .....                                             | 12 |
| 8 Källförteckning .....                                        | 13 |

### 3 Terminologi

Nedan följer en förteckning av olika uttryck, termer och förkortningar med korta förklaringar som nämns i denna rapport.

|                               |                                                                                                                                                  |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API</b>                    | Står för "Application Programming Inter-face" och används för att hämta extern data och information som sedan kan användas i webbapplikationen.  |
| <b>JSON-data</b>              | Ett textbaserat format som används för att strukturera och överföra data mellan exempelvis ett API och en webbapplikation.                       |
| <b>Asynkron programmering</b> | Genom att ange async och await i JavaScript funktioner så kan man hantera API-anrop utan att stoppa användargränssnittet.                        |
| <b>DOM-manipulation</b>       | Används för att dynamiskt kunna ändra användargränssnittet baserat på användarens interaktion på webbapplikationen.                              |
| <b>LocalStorage</b>           | Används för att kunna spara information lokalt på användarens webbläsare. Den informationen kan sedan användas för att visa upp det som sparats. |
| <b>API-anrop</b>              | En förfrågan från applikationen till ett API för att hämta eller skicka information.                                                             |
| <b>Rendering</b>              | Processen där data omvandlas till visuellt innehåll i webbapplikationens användargränssnitt.                                                     |

## 4 Inledning

I dagens digitala samhälle finns det ett otroligt stort utbud av filmer och TV-serier tillgängligt via olika plattformar och streamingtjänster som exempelvis Netflix och Viaplay. Den växande digitala närvaron har gjort det väldigt enkelt att konsumera digitalt innehåll. Trots enkelheten så kan det ibland vara svårt att hålla koll på vilka filmer man vill se och vilka man redan har sett. Syftet blev alltså för detta projekt att utveckla och skapa en webbapplikation som hjälper användaren att hitta och spara filmer och TV-serier. Denna rapport följer de olika stegen som gjorts för att skapa en sådan webbapplikation – från planering till färdigt slutresultat.

Rapporten utgår ifrån den fiktiva kunden Watchly, ett nyetablerat företag med ambitionen att ta sig in på marknaden för digitala filmtjänster.

### 4.1 *Bakgrund och problemmotivering*

I takt med den allt växande digitala närvaron i dagens samhälle så återstår ett återkommande problem varje fredagskväll när man ska se på TV - vad ska man titta på ikväll? Det stora utbudet riskerar att leda till massor med bläddring mellan filmer utan att riktigt kunna fatta ett beslut om vad man faktiskt vill titta på [1]. Utöver detta så finns det begränsade möjligheter att organisera och anpassa sitt innehåll på ett mer personligt vis. Funktioner för att spara, kategorisera och skapa egna listor över filmer och serier kan upplevas som otillräckliga.

Problemmotiveringen i detta fall, med ovan i åtanke, är att skapa en webbapplikation som hjälper användaren att upptäcka nytt innehåll men även att organisera sitt tittande. Applikationen ska göra det möjligt för användaren att söka efter filmer och TV-serier samt spara dessa i en personlig lista. Vilket möjliggör det för användaren att få en bättre överblick över vilka filmer som användaren tycker är intressanta.

### 4.2 *Konkreta mål*

Nedan följer konkreta mål för projektet:

- Användaren ska kunna söka efter filmer och TV-serier samt sortera efter genre
- Användaren ska kunna spara filmer i en personlig lista
- Webbapplikationen ska hela tiden visa dem senaste och populäraste titlarna
- Minst två API:er ska användas och kommunicera med varandra

## 5 Metod

Utvecklingen av denna webbapplikation har skett i olika steg. Det första steget var en planeringsfas där det fiktiva företags grafiska profil togs fram och skisser tillverkades i programvaran Figma. Detta gjordes för att kunna ha något att luta sig mot igenom arbetet, och har varit till stor nytta.

Efter planeringsfasen påbörjades sökningen efter vilka API:er som skulle användas. API står för "Application Programming Interface" och används för att hämta externa data och information som sedan kan användas i webbapplikationen. För att hämta information om filmer och TV-serier valdes The Movie Database – TMDb [2]. TMDb användes som huvudsaklig källa för information om filmer och TV-serier, exempelvis titlar, beskrivningar, genrer, betyg, skådespelare och liknande titlar. Sedan valdes YouTube API [3] för att kunna visa upp trailers.

När detta valts så var det dags att påbörja utvecklingen av projektet och utvecklingsmiljön som användes var Visual Studio Code och applikationen utvecklades med HTML, JavaScript och CSS, där Sass användes som CSS-preprocessor. Vite användes som utvecklingsverktyg för att strukturera och köra projektet. API-anrop görs via separata funktioner som hämtar film- och seriedata från TMDb. Dessa data bearbetas innan de skickas vidare till renderingsfunktioner. Filmer och serier renderas dynamiskt i DOM genom funktioner som skapar HTML-element baserat på API-data.

För att användaren skulle kunna hitta relevant innehåll användes en kombination av sökning och filtrering. De filmer och TV-serier som hämtats från API:erna sparades tillfälligt i en array i applikationen. Denna array användes sedan som underlag för sökfunktionen och för filtrering efter mediatyp och genre.

Filtreringen byggdes med olika variabler som höll reda på det aktuella valet, exempelvis om användaren valt film eller TV-serie och vilken genre som valts. När användaren ändrade ett filter filtrerades den sparade datan på nytt och resultatet renderades om. På detta sätt kunde flera filter kombineras utan att ett nytt API-anrop behövde göras för varje klick.

För att hantera användarens personliga watchlist användes webbläsarens LocalStorage. På så sätt kunde sparade filmer, status, betyg och kommentarer lagras lokalt och finnas kvar även när sidan laddades om. Filtrering och sökning implementerades genom att den hämtade datan bearbetades i JavaScript och därefter renderades på nytt utifrån användarens val.

## 6 Konstruktion

### 6.1 *Designskisser och grafisk profil*

Utvecklingen av webbapplikationen har skett i olika skeden. Under planeringsfasen var målet att skapa en modern och tilltalande webbplats för att presentera och organisera filmer och TV-serier. Många streamingtjänster använder mörka gränssnitt, vilket även valdes för denna applikation för att filmbilderna skulle framträda tydligare mot bakgrunden. Den röda färgen valdes med inspiration från biosalonger, där röda stolar är ett vanligt inslag, och fick därför speglas i logotypen och webbplatsens grafiska profil. Den gröna och blå färgen för tankarna till neonbelysning och användes för att skapa en känsla av lysande färger mot den mörka bakgrunden.

När den grafiska profilen var färdig övergick planeringen till att skissa upp webbplatsens struktur och olika vyer. Under arbetet har en iterativ process använts, där olika delar av applikationen har utvecklats, testats och sedan förändrats när nya behov eller förbättringsmöjligheter upptäckts. Den ursprungliga skissen skiljer sig därför från den färdiga applikationen.

Den största förändringen återfinns i webbplatsens header. I den ursprungliga skissen fanns en hero-bild och navigeringsfältet hade en tydligare färg mot den mörka bakgrunden. I den färdiga applikationen har hero-bilden tagits bort och navigeringen placerats längst upp på sidan med ett mer diskret uttryck. Hover-effekter används för att tydliggöra vilka element som går att interagera med.

Webbplatsens HTML-struktur skapades först för navigationsfältet och de olika sektionerna där film- och seriedata senare skulle renderas. Därefter utvecklades designen parallellt med JavaScript-funktionaliteten. CSS skrevs med SASS, vilket gjorde det möjligt att dela upp stilreglerna i flera filer och strukturera projektets design på ett mer organiserat sätt. SASS användes även för exempelvis mixins och återanvändbara stilregler.

### 6.2 *Hämtning och hantering av data*

En viktig del av konstruktionen var att hitta en webbtjänst som kunde tillhandahålla det film- och serieinnehåll som applikationen behövde. Flera alternativ undersöktes under arbetet, men den webbtjänst som slutligen valdes var The Movie Database (TMDB). TMDB tillhandahåller bland annat information om populära filmer och TV-serier, genrer, skådespelare, beskrivningar, liknande titlar och trailers.

För att hämta informationen används funktionen `fetchMovies()`. Funktionen ansvarar för själva API-anropet och returnerar den data som hämtas från TMDB. Olika endpoints används beroende på vilken typ av innehåll som ska hämtas. Exempelvis används ett anrop för populära filmer medan ett annat används för TV-serier eller filmer inom en viss genre.

Under utvecklingen har API-anropen justerats flera gånger för att begränsa och anpassa resultatet. Exempelvis anges att endast engelskspråkiga titlar ska hämtas och att resultatet ska sorteras efter popularitet. För TV-serier behövdes separata anrop eftersom TMDB använder andra endpoints och genre-ID:n för serier än för filmer.

När data hämtas från TMDB kompletteras objekten även med egenskapen `media_type`. För filmer sätts värdet till "movie" och för TV-serier till "tv". Detta gör det möjligt att senare skilja mellan de två innehållstyperna när användaren filtrerar innehållet.

### **6.3    *Rendering av film- och seriekort***

När data har hämtats från TMDB skickas den vidare till funktionen `displayMovies()`. Funktionen används för att återanvända samma struktur när olika typer av film- och seriedata ska visas på sidan.

Funktionen tar emot tre argument:

`displayMovies(movies, sectionID, filter)`

`movies` innehåller filmerna eller serierna som ska visas, `sectionID` anger vilken HTML-sektion som ska fyllas och `filter` anger hur många titlar som ska renderas.

På så sätt kan samma funktion exempelvis användas för sektionen med populära filmer och för sektionen där användaren själv filtrerar mellan filmer och TV-serier. Funktionen skapar dynamiskt HTML-element för varje titel och lägger bland annat till filmens poster, en overlay och ikoner för Watchlist och mer information.

En viktig del av denna funktion är också att information från användarens Watchlist läggs tillbaka på de hämtade objekten genom funktionen `addJournalData()`. Detta behövs eftersom filmdata som hämtas från TMDB inte innehåller användarens egna uppgifter, exempelvis status, rating och kommentarer.

### **6.4    *Detaljerad information och modal***

När användaren klickar på informationsikonen behöver mer information än den som visas på filmkortet hämtas. Då anropas `fetchMovieDetails()`.

Funktionen hämtar kompletterande information för den valda titeln, exempelvis skådespelare, trailers och liknande titlar. Informationen returneras sedan till den funktion som ska visa modalen. För att rendera modalen anropas funktionen `displayModal()`. Den tar emot informationen och bygger upp modalens HTML. Där visas bland annat titel, mediatyp, betyg från TMDB, beskrivning, skådespelare, möjlighet att lägga till titeln i Watchlist samt möjlighet att spela upp trailer.

För att få fram trailers så används den information om titeln som kommer från TMDB för att kunna göra ett anrop till YouTube för att kunna få fram en relevant video.

### **6.5    *Watchlist***

En central del av projektet var att användaren skulle kunna skapa och organisera en egen lista med filmer och serier. För att uppnå detta placerades en bokmärkesikon på varje filmkort. När användaren klickar på ikonen kontrolleras först om filmen redan finns sparad. Om filmen inte finns i Watchlist sparas den tillsammans med grundinformationen från TMDB och kompletteras med egna egenskaper för användaren.

På så sätt innehåller objektet både information från den externa webbtjänsten och information som användaren själv har lagt till.

Filmerna sparas i webbläsarens `localStorage`. `LocalStorage` valdes eftersom projektet inte kräver någon inloggning eller extern databas. Informationen kan därför sparas lokalt i användarens webbläsare och finnas kvar även när sidan laddas om.



Funktionerna `getSavedMovies()` och `saveToLocalStorage()` används för att läsa respektive skriva information till `localStorage`. När en film läggs till eller tas bort uppdateras `localStorage` och `Watchlist` renderas sedan om.

På `Watchlist`-sidan används `displayWatchlist()` för att visa de sparade titlarna. Filmkorten liknar de på startsidan, men har en något annorlunda funktionalitet. Bokmärkesikonen används här för att ta bort filmen från listan istället för att lägga till den på nytt.

## 6.6 *Movie Journal*

`Watchlist`-funktionen utvecklades vidare eftersom ett av projektets mål var att användaren inte bara skulle kunna spara filmer utan även sortera och organisera sitt sparade innehåll. Därför skapades en `Movie Journal` som är kopplad till varje sparad titel.

När användaren befinner sig på `Watchlist`-sidan och klickar på informationsikonen öppnas en separat modal genom funktionen `displayMovieJournal()`. Denna modal innehåller fortfarande grundläggande information om filmen, men kompletteras med funktioner där användaren kan dokumentera sina egna tankar.

Användaren kan bland annat ange:

- **Status:** Vill se, måste se, påbörjad eller sedd
- **Rating:** Ett eget betyg mellan ett och fem stjärnor
- **Kommentar:** En egen kommentar om filmen

Ratingen är uppbyggd med fem stjärnor. När användaren för muspekaren över en stjärna fylls den stjärnan och de tidigare stjärnorna i visuellt. När användaren klickar sparas ratingen och de valda stjärnorna förblir aktiva.

När användaren klickar på knappen för att spara `Movie Journal` letar JavaScript efter den aktuella filmen i de filmer som finns sparade i `localStorage`. Därefter uppdateras det befintliga objektet med användarens nya information.

## 6.7 *Kopplingen mellan TMDB-data och användarens egna data*

En utmaning med lösningen var att informationen från `TMDB` och användarens egna information kommer från två olika håll. `TMDB` innehåller exempelvis titel, poster, genre och beskrivning, medan `localStorage` innehåller exempelvis status, rating och kommentar.

För att kunna visa den egna informationen även på startsidan används `addJournalData()`. Funktionen går igenom de filmer som hämtats från `TMDB` och letar efter en motsvarande film i användarens sparade filmer. Om filmen finns i `Watchlist` kompletteras `TMDB`-objektet med användarens sparade information.

Detta gör att samma film kan visas på startsidan med exempelvis statusen "Sedd" på filmkortet trots att själva informationen om statusen inte kommer från `TMDB`.

Funktionen `displayMovies()` anropar `addJournalData()` innan filmerna renderas. Därmed kan informationen från `Movie Journal` återanvändas på flera platser i applikationen utan att användaren behöver öppna `Watchlist`-sidan för att se sin status.

De centrala funktionerna har därmed olika ansvarsområden men bygger på varandra. `fetchMovies()` ansvarar för att hämta det grundläggande film- och serieinnehållet från TMDB, medan `displayMovies()` ansvarar för att presentera innehållet i applikationens olika sektioner. När användaren vill se mer information används `fetchMovieDetails()` för att hämta kompletterande information, som sedan skickas till `displayModal()`.

Watchlist-funktionaliteten använder `localStorage` för att spara användarens filmer. `displayWatchlist()` används för att presentera dessa filmer på Watchlist-sidan och `displayMovieJournal()` ger användaren möjlighet att komplettera de sparade filmerna med egen information. Genom `addJournalData()` kan den sparade informationen sedan kopplas tillbaka till de filmer som hämtas från TMDB och visas även på startsidan.

## Resultat

Resultatet av projektet är en fungerande webbapplikation för att söka efter, upptäcka och organisera filmer och TV-serier. Applikationen använder TMDB som källa för film- och seriedata och presenterar informationen i ett gränssnitt där användaren kan söka, filtrera och spara titlar. De uppsatta målen har uppnåtts genom de funktioner som implementerats i applikationen.

Det första målet var att användaren ska kunna söka efter filmer och TV-serier samt sortera innehållet efter genre. Detta har implementerats genom en sökfunktion där användaren kan skriva in en titel och få fram matchande resultat. Sökningen omfattar både filmer och TV-serier och resultaten presenteras separat från webbplatsens övriga innehåll.

Utöver sökfunktionen finns även möjlighet att filtrera innehållet efter mediatyp och genre. Användaren kan välja mellan filmer och TV-serier och därefter välja en genre.

Det andra målet var att användaren ska kunna spara filmer i en personlig lista. Detta har implementerats genom en bokmärkesfunktion som finns på film- och seriekorten samt i den detaljerade informationsrutan. När användaren sparar en titel läggs den till i en personlig Watchlist. De sparade titlarna lagras i webbläsarens localStorage. Det gör att informationen finns kvar när sidan laddas om och gör det möjligt att använda Watchlist utan att en separat databas eller användarinloggning behövs.

Watchlist-funktionen har även utökats med en Movie Journal. För varje sparad titel kan användaren ange en egen status, sätta ett eget betyg mellan ett och fem stjärnor samt skriva en kommentar om filmen.

Ett av de andra målen var att minst två API:er ska användas och kommunicera med varandra. Projektet använder TMDB och YouTube.

TMDB används som den huvudsakliga källan för film- och seriedata. När användaren öppnar den detaljerade informationsrutan hämtas bland annat information om skådespelare, liknande titlar och trailerinformation. Med hjälp av titelinformation från TMDB så kan ett anrop göras till YouTube API för att få fram en video. Denna information används sedan för att bädda in trailern från YouTube i applikationens modal.

API:erna används därmed i kombination för att skapa en mer komplett presentation av den valda titeln. TMDB står för informationen om själva filmen eller TV-serien medan YouTube används för att tillgängliggöra videoinnehållet.

Sammanfattningsvis uppfyller den färdiga webbapplikationen de funktionella mål som sattes upp i projektets inledning. Användaren kan upptäcka aktuella filmer och TV-serier, söka och filtrera bland innehållet samt skapa och organisera en personlig Watchlist med hjälp av Movie Journal.

## 7 Slutsatser

Under projektets gång har det blivit tydligare hur data från en extern webbtjänst kan hämtas, bearbetas och sedan omvandlas till ett användargränssnitt genom JavaScript. Det har även gett en bättre förståelse för hur olika funktioner behöver samverka för att skapa ett sammanhängande flöde genom hela applikationen.

Det har varit många utmaningar med detta projekt. Ett av de första hindren som uppstod var filtrering och sortering av filmer och serier. För att kunna skilja filmer och serier åt så behövdes det läggas till information i filmobjektet. Det krävdes flera iterationer för att få logiken stabil och undvika problem som dubletter, felaktig filtrering och att gränssnittet inte uppdaterades på rätt sätt.

Även arbetet med Watchlist och Movie Journal innebar flera utmaningar. Till skillnad från den övriga informationen kommer användarens status, rating och kommentarer inte från TMDB utan behöver sparas lokalt. Det innebar att data från API:et behövde kombineras med användarens egna data från localStorage. En särskild utmaning var därför att se till att den egna informationen följde med filmen även när samma film renderades på olika delar av webbplatsen. Funktionen `addJournalData()` blev en lösning på detta genom att komplettera den data som hämtas från TMDB med den information som användaren tidigare sparar.

Om projektet hade vidareutvecklats finns det flera funktioner som skulle kunna förbättra applikationen ytterligare. Den största utvecklingsmöjligheten är att ersätta localStorage med en databas kopplad till användarkonton. I den nuvarande lösningen är Watchlist knuten till den webbläsare där informationen sparas. Med användarkonton och molnlagring skulle användaren kunna logga in och komma åt sin Watchlist och Movie Journal från olika enheter.

Sökfunktionen skulle också kunna utvecklas ytterligare. Den nuvarande sökningen bygger huvudsakligen på att matcha användarens sökord mot titlarnas namn. En mer avancerad sökfunktion skulle exempelvis kunna kombinera fritextsökning med genre, mediatyp, betyg, releaserdatum och popularitet. Det skulle ge användaren större möjlighet att hitta en specifik typ av film eller serie.

En ytterligare utvecklingsmöjlighet är att implementera infinite scroll istället för att begränsa antalet titlar som visas åt gången. I den nuvarande lösningen renderas ett begränsat antal titlar för att hålla gränssnittet överskådligt. Med infinite scroll skulle nya resultat kunna hämtas och renderas automatiskt när användaren närmar sig slutet av listan. Detta skulle dock även kräva ytterligare hantering av API-anrop och paginering.

Sammanfattningsvis har projektet gett praktisk erfarenhet av att bygga en större frontend-applikation där flera funktioner och datakällor behöver samverka. Framför allt har arbetet med state, dynamisk rendering, API-anrop och localStorage bidragit till en större förståelse för hur JavaScript kan användas för att skapa ett interaktivt gränssnitt. Projektets iterativa arbetssätt har också visat att en fungerande lösning ofta behöver utvecklas och omprövas flera gånger innan olika delar fungerar tillsammans på ett stabilt sätt.

## 8 Källförteckning

- [1] Wikipedia, "The Paradox of Choice",  
[https://en.wikipedia.org/wiki/The\\_Paradox\\_of\\_Choice](https://en.wikipedia.org/wiki/The_Paradox_of_Choice)  
Hämtad: 2026-05-04
- [2] The Movie Database, API documentation, <https://developer.the-moviedb.org/docs/getting-started>  
Hämtad: 2026-08-16
- [3] YouTube API, API documentation <https://developers.google.com/youtube/v3/getting-started>  
Hämtad: 2026-08-16